PROJECT OVERVIEW / TECHNICAL GUIDE

AgentWard + MuseVerify

AI-assisted remediation with mandatory security checks.

A repair earns trust through evidence.

Inspect the source. Reproduce the exploit. Review the repair. Verify its behavior. Preserve the audit trail.

7	5	3
security phases	ensemble modes	maximum concurrent sandboxes

Purpose, objectives, architecture, use cases, file hierarchy, deployment and recorded verification.

30 October 2026 | Repository snapshot 99ac51a

github.com/ephraimcrypto21/AgentWard-MuseVerify

01 / PROJECT GUIDE

Purpose and objectives

A security harness for turning Python vulnerability findings into reviewable, behaviorally verified candidate repairs.

AgentWard + MuseVerify brings runtime protection and remediation verification into one workflow. AgentWard screens incoming material and proposed actions. MuseVerify reproduces vulnerabilities, generates repairs and tests whether those repairs preserve useful behavior while resisting attacks.

What problem does it address?

Problem	Project response
An AI-generated patch looks convincing but does not fix the exploit.	Require exploit replay, independent review, CodeShield and post-patch regression.
Attacker-controlled source text contains instructions for the agent.	Scan the complete input with Prompt-Guard and rescan sanitized content before model use.
Static alerts include benign or unreachable behavior.	Use sandbox observations to distinguish confirmed exploits, tested false positives and inconclusive runs.
A model or infrastructure failure silently lowers confidence.	Keep Meta mandatory, verify optional providers and deny verification when required evidence is missing.

Core objectives

+ **Protect the workflow.** Screen secrets and actions; detect attempts to bypass review or drift outside the authorized task.

+ **Verify the repair.** Require agreement and executable evidence before publishing a remediation result.

+ **Preserve functionality.** Test normal inputs as well as exploit and decoy inputs.

+ **Make decisions reviewable.** Record model identities, votes, traces, hashes, diffs, outcomes and available usage metrics.

+ **Help contributors investigate further.** Map cross-file flows and turn unsupported assumptions into explicit review assertions.

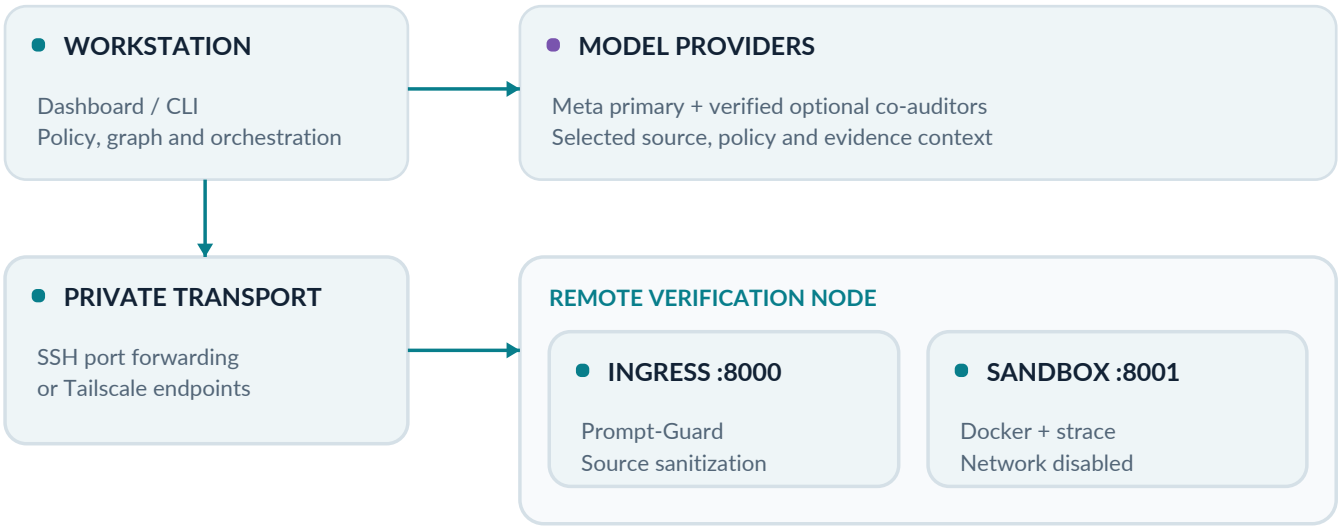
> **Intended scope:** a Python-focused security and remediation prototype with a controlled benchmark corpus. Its recorded results demonstrate the implemented workflow; they are not a universal security guarantee.

Source: README.md; main_pipeline.py

02 / PROJECT GUIDE

System architecture

The workstation coordinates reasoning and policy. A private remote node performs ingress classification and isolated execution.



Results return to the orchestrator for gated decisions and report export.

Boundary	Responsibility
Workstation	FastAPI dashboard or CLI, provider configuration, session policy, static graph analysis and report assembly.
Model APIs / CLI	Meta leads synthesis and deep analysis. Verified optional providers independently inspect selected task context.
Remote ingress	Prompt-Guard evaluates the complete source in overlapping windows and verifies the sanitized result.
Remote Docker execution	The target and deterministic harness run with networking disabled. strace and behavioral probes supply execution evidence.

How data moves

Source and policy context enter the orchestrator. It requests ingress screening, prepares a behavioral harness and sends the target to the sandbox. Model reviews receive selected source and execution evidence. Successful candidates become separate patch artifacts; benchmark originals remain unchanged.

> **Parallel work:** during Phase 2, graph construction and policy preparation can overlap sandbox execution. Preparation does not authorize the next action. Both task scheduling and sandbox concurrency are bounded at three.

Source: dashboard.py; main_pipeline.py; ai_provider.py; sandbox_server.py

03 / PROJECT GUIDE

The seven security phases

Every phase answers a different question. An approval at one phase cannot replace the evidence required at the next.

Phase	Question and activity	Required result
1 Ingress Shield	Does source contain injected instructions? Scan all input windows and sanitize detected instruction material.	Valid source plus verified Prompt-Guard coverage and a clean rescan.
2 Syscall Sandbox	Can the suspected vulnerability be exercised? Import the target and run behavioral probes in Docker.	Completed probes, kernel strace evidence and matching submitted source/harness hashes.
3 Runtime Hooks	May the workflow continue under the current policy? Run deterministic checks, ensemble review and trajectory monitoring.	Current-policy approval. Denial or detected drift triggers workspace rewind.
4 Patch Verification	Does the candidate repair the issue without an obvious new hazard? Meta synthesizes; independent models, AST checks and CodeShield review.	All applicable gates pass; public signatures and a reachable tripwire are preserved.
4.5 Adversarial Self-Play	Does the candidate survive mutated and multi-step scenarios? Compare each attack with an original-code control.	No surviving viable exploit; valid replay evidence. Unsupported scenarios are not counted as wins.
5 Regression	Does the original exploit stay blocked while normal behavior and decoy detection work?	Functional checks pass, exploit replay fails and the tripwire fires for its decoy.
5.1 Semantic Hunter	Where else might the root cause or unsafe assumption exist? Analyze related files with Wonder and active co-auditors.	Static and model findings reported with coverage limits, separately from runtime proof.

What REMEDIATED means

A candidate passed the mandatory repair gates and verified regression. The project writes a separate candidate file and exact diff; it does not automatically apply the patch to the source corpus or merge a pull request.

> **Fail closed:** incomplete traces, unavailable required checks, malformed votes and stale policy approvals cannot establish a verified remediation or a verified benign result.

Source: main_pipeline.py; agentward_stage1.py; museverify_stage2.py; museverify_stage4.py

04 / PROJECT GUIDE

Providers and operator control

Meta remains the primary engine. Optional reviewers participate only after a successful preflight check.

muse-spark-1.3 leads patch synthesis and runtime inspection. **muse-wonder-1.0** leads semantic and reachability analysis, and provides native independent review when needed. Custom BYOK configuration uses the model supported by the chosen endpoint.

Mode	Participating topology
auto	Meta plus every healthy optional provider.
all_three	Meta + Anti-Gravity + Custom BYOK, with explicit downgrade notices if a secondary is unavailable.
meta_plus_agy	Meta plus verified Anti-Gravity.
meta_plus_custom	Meta plus verified Custom BYOK.
meta_only	Spark and Wonder cross-checking each other.

Provider health is part of the workflow

Preflight checks run concurrently. Empty or placeholder BYOK configuration, authentication failures, invalid content and timeouts exclude that provider. Anti-Gravity can cascade through configured high-end candidates after quota or availability failures. A selected reviewer's later failure denies that review.

Mid-run steering

```
POST /api/steer
{"constraint":"Forbid subprocess calls; enforce pathlib commonpath"}
```

An operator can add restrictions without restarting the batch. Session policy revisions increase monotonically; a change invalidates an in-flight approval. Recognized subprocess and path restrictions also receive AST enforcement. General natural-language constraints are reviewed by the ensemble.

> **Control boundary:** steering tightens restrictions; it cannot waive Prompt-Guard, CodeShield, sandbox evidence or consensus. The trajectory monitor checks for goal drift, covert environment probing and review-bypass attempts.

Source: ai_provider.py; agentward_stage3.py; session_policy.py

05 / PROJECT GUIDE

Use cases and practical value

The strongest current fit is controlled Python security analysis where the target can be exercised by the available harness.

Use case	How the project helps	Expected output
Application-security triage	Reproduce a suspected SQL or shell injection instead of relying only on a static pattern.	Confirmed exploit, tested false positive or inconclusive outcome.
AI-assisted repair review	Generate a candidate and subject it to independent reviewers, static checks and attack replay.	Verified candidate diff or a recorded rejection.
Agent guardrail evaluation	Exercise secret screening, policy changes, malformed votes and trajectory denials.	Auditable decisions and rewind behavior.
Security research and demonstrations	Compare attack mutations with original-code controls and inspect a replayable phase timeline.	Scenario results, traces, metrics and benchmark reports.
Private remote verification	Run resource-heavy ingress classification and Docker/strace on a separate node via SSH or Tailscale.	Private service endpoints and remote execution evidence.

Example: SQL injection

A query function builds SQL with user-controlled text. The harness first demonstrates the breakout. Meta proposes parameter binding, the reviewers assess the change, and replay checks the original exploit, normal lookups, mutated inputs and decoy logging. Related SQL paths become semantic-hunt leads.

Example: command injection

A helper passes a user-controlled string to a shell. The workflow confirms the shell escape, evaluates a candidate with a safer process boundary, and checks the original return contract. Normal commands and decoy inputs are verified separately from attack inputs.

> **Integration opportunity:** SARIF and machine-readable results can support a future CI review step. The repository does not currently provide a complete hosted CI service, automatic pull-request merge workflow or support for arbitrary programming languages.

Source: README.md; benchmark_targets/; adversarial_selfplay.py

06 / PROJECT GUIDE

Repository file hierarchy

A map of the source snapshot. Repeated benchmark and test files are grouped; generated runtime folders are listed separately.

```
AgentWard-MuseVerify/
|-- README.md                      # Setup and project overview
|-- .env.example                   # Provider and endpoint template
|-- .agents/hooks.json             # PreToolUse hook configuration
|-- main_pipeline.py               # Shared CLI/dashboard orchestration
|-- dashboard.py                   # HTTP and WebSocket controls
|-- templates/index.html           # Dashboard UI
|-- ai_provider.py                 # Routing, preflight, votes and usage
|-- session_policy.py              # Additive session restrictions
|-- agentward_stage1.py            # Ingress client and evidence checks
|-- ingress_sanitizer.py           # Source-preserving sanitization
|-- ingress_server.py              # Remote Prompt-Guard service
|-- agentward_stage3.py            # Policy review, monitor and rewind
|-- enforce_permissions.py         # Command permission hook
|-- museverify_stage2.py           # Harness, sandbox client and triage
|-- sandbox_server.py              # Remote Docker/strace service
|-- museverify_stage4.py           # Candidate synthesis and patch gates
|-- adversarial_selfplay.py        # Mutations and multi-step replay
|-- security_graph.py              # Static import/call/taint analysis
|-- semantic_hunter.py             # Root cause and related-file review
|-- reporting.py                   # Diffs, metrics, SARIF and briefs
|-- poisoned_app.py                # Root demonstration target
|-- benchmark_targets/
|   |-- cwe_89_sqli.py             # SQL injection
|   |-- cwe_78_cmd.py              # Command injection
|   |-- cwe_338_crypto.py          # Weak randomness
|   |-- fp_01.py ... fp_20.py      # False-positive controls
|   +-- poison_docstring.py, poison_variable.py
|-- tests/                         # Unit, boundary, browser and E2E tests
|   +-- schemas/                   # Vendored SARIF 2.1.0 schema
|-- systemd/                       # Two remote service unit files
|-- deploy_remote_services.sh      # Remote service/image deployment
|-- requirements.txt              # Local application dependencies
|-- requirements-dev.txt          # Test and development tools
|-- requirements-node.txt         # Remote node dependencies
+-- docs/VERIFICATION.md           # Recorded validation and limitations
```

Generated and ignored: .env contains local credentials; venv/ contains installed dependencies; worktrees/ holds temporary workspaces; patches/ holds candidates and diffs; reports/ holds telemetry, SARIF, briefs and run archives. These are distinct from tracked source.

Source: Tracked repository tree at 99ac51a; .gitignore

07 / PROJECT GUIDE

Module responsibilities and outputs

Start with the orchestrator, then follow the evidence boundary you want to understand or extend.

Area / files	Responsibility
main_pipeline.py dashboard.py templates/index.html	Coordinate all seven phases; run single or bounded batch audits; stream run-tagged events and expose operator controls.
ai_provider.py session_policy.py	Discover healthy providers, select a topology, preserve task context and usage evidence, and track additive restrictions.
agentward_stage1.py ingress_server.py ingress_sanitizer.py	Classify and sanitize incoming source. Reject missing or incomplete ingress evidence.
agentward_stage3.py enforce_permissions.py .agents/hooks.json	Screen proposed actions, obtain policy votes, inspect trajectories and rewind managed workspaces. The hook file declares integration with a compatible agent host.
museverify_stage2.py sandbox_server.py	Build executing probes, invoke remote isolation, normalize syscall evidence and distinguish exploitability from inconclusive execution.
museverify_stage4.py adversarial_selfplay.py	Synthesize candidates through bounded refinement, enforce patch gates and replay adversarial scenarios with positive controls.
security_graph.py semantic_hunter.py	Resolve bounded cross-file flows and assumptions, then combine static findings with deep model review.
reporting.py	Create exact diffs and atomically export machine-readable metrics, SARIF and executive briefs.

Artifacts that matter

patches/<run-id>/ - accepted candidate source and its unified diff.
reports/benchmark_metrics.json - outcomes, provider votes, usage and timings.
reports/agentward_sarif_report.sarif - security findings and verified fixes.
reports/executive_brief.md / .html - evidence summary and unresolved assertions.
reports/runs/ - per-run archives, including terminal failures.

> **Reading order:** README.md > main_pipeline.py > the phase module > its tests > docs/VERIFICATION.md. Test fixtures and dry-run entry points are explicitly simulated.

Source: Module docstrings and entry points; reporting.py

08 / PROJECT GUIDE

Getting started

Provision the workstation and the remote verification node before launching a live audit.

1. Prepare the workstation

Use Python 3.12 or newer as recommended by the repository. Install application and development dependencies, then populate the local environment template.

```
python3 -m venv venv
. venv/bin/activate
python -m pip install -r requirements-dev.txt
cp .env.example .env
```

Configure META_API_KEY and, if needed, META_API_BASE_URL. Select ENSEMBLE_MODE. Optional Anti-Gravity configuration uses AGY_BIN and AGY_SELECTED_MODEL; optional BYOK configuration uses CUSTOM_API_URL, CUSTOM_API_KEY and CUSTOM_MODEL_NAME. Use the chosen endpoint's supported model identifier.

2. Prepare the remote node

The Ubuntu node needs Docker, a Python environment with the packages in requirements-node.txt, and provisioned Prompt-Guard-86M weights. The classifier currently loads Niansuh/Prompt-Guard-86M. Unavailable weights fail closed.

Place the ingress server, sanitizer, sandbox server and deployment script in /home/ubuntu/agentward-node/. The deployment script installs the two systemd services and builds a strace-capable sandbox image when needed. The default services listen on loopback ports 8000 and 8001.

3. Connect and launch

Choose one of the connection methods on the next page. Set both endpoint URLs in the workstation's .env, then start the dashboard.

```
python -m uvicorn dashboard:app --host 127.0.0.1 --port 8080

# Alternatively, invoke the shared workflow from the CLI:
python main_pipeline.py poisoned_app.py
python main_pipeline.py benchmark_targets
```

Open http://127.0.0.1:8080. Check provider status, select a target and inspect each phase and its evidence. Keep the dashboard local or behind authenticated access controls.

> **Normal operating sequence:** health check > provider preflight > target selection > audit > evidence review > deliberate patch application. The original benchmark remains intentionally vulnerable for repeatable controls.

Source: README.md; requirements*.txt; deploy_remote_services.sh

09 / PROJECT GUIDE

SSH and Tailscale endpoints

Both methods connect the workstation to the same ingress and sandbox APIs while the remote services continue to bind to loopback.

Option A: SSH forwarding

Run this from the workstation, replacing YOUR_HOST with the node's reachable address.

```
ssh -N -L 18000:127.0.0.1:8000 \
  -L 18001:127.0.0.1:8001 ubuntu@YOUR_HOST

# Workstation .env
AGENTWARD_INGRESS_URL=http://127.0.0.1:18000/scan-ingress
AGENTWARD_SANDBOX_URL=http://127.0.0.1:18001/execute
```

Option B: private Tailscale endpoints

Connect both machines to the same tailnet. On the remote node, use Tailscale Serve to forward private ports to the existing services.

```
tailscale ip -4
sudo tailscale serve --bg --tcp=8000 tcp://127.0.0.1:8000
sudo tailscale serve --bg --tcp=8001 tcp://127.0.0.1:8001
tailscale serve status
```

Replace NODE_TAILSCALE_IP with the printed address, or use the node's MagicDNS name when enabled. Set the workstation's .env to:

```
AGENTWARD_INGRESS_URL=http://NODE_TAILSCALE_IP:8000/scan-ingress
AGENTWARD_SANDBOX_URL=http://NODE_TAILSCALE_IP:8001/execute
```

Check /health on both chosen service URLs. Tailscale carries this HTTP traffic inside its encrypted network, and Serve exposes the ports within the tailnet. Restrict access to authorized operators through tailnet grants or ACLs. Restart the dashboard after changing endpoint URLs.

> **Verification scope:** the recorded acceptance used SSH forwards. Tailscale setup is documented as an alternative; this guide does not claim that a new Tailscale deployment was tested during PDF generation.

Source: README.md; systemd/ service units

10 / PROJECT GUIDE

Recorded verification

Acceptance recorded on 30 October 2026 demonstrates three verified repairs and a benign control across the live workflow.

46
automated tests passed

82%
statement coverage

4 / 4
live target outcomes matched

Target	Recorded result	Evidence summary
poisoned_app.py	REMEDIATED	Instruction injection sanitized; SQL repair passed all applicable gates.
cwe_89_sqli.py	REMEDIATED	SQL injection repair passed independent review and regression.
cwe_78_cmd.py	REMEDIATED	Command injection repair preserved normal behavior and passed exploit replay.
fp_01.py	FALSE_POSITIVE	Completed benign sandbox result and runtime policy review; no patch generated.

Each of the three repairs neutralized three viable adversarial mutations, for nine recorded neutralizations in total. Regression exercised normal inputs and live decoy guards. The combined SARIF validated against the official vendored 2.1.0 schema, and all three diffs passed git apply --check.

Additional checks

+ **Evidence boundaries.** Tests cover missing traces, mismatched hashes, unavailable classification, malformed votes, credential changes, stale approvals and explicit simulation separation.

+ **User interface.** Browser checks covered seven phase cards, five modes, steering, both diff layouts and executive/taint views with zero page errors.

+ **Provider acceptance.** Meta's two native engines and Anti-Gravity were online. The configured optional BYOK endpoint returned HTTP 401 and was excluded before task use.

> **How to interpret the numbers:** 82% covers 2,235 statements in imported application modules. It is not security coverage. Four live targets are a selected acceptance set, not a claim that the entire benchmark corpus or arbitrary projects are proven safe.

Compilation, Ruff correctness checks, dependency consistency and source-diff checks also passed. Earlier rejected or inconclusive runs were retained locally. The final results are recorded in docs/VERIFICATION.md; no new full pipeline run was performed to create this guide.

Source: docs/VERIFICATION.md; recorded acceptance, 30 October 2026

11 / PROJECT GUIDE

Operations, scope and next steps

Use machine-readable outputs for review, and treat uncovered behavior as a follow-up task rather than a verified result.

Interface	Purpose
GET /api/health GET /api/providers	Read running state, provider preflight status and selected topology.
POST /api/run POST /api/batch	Launch a target audit or a bounded benchmark batch.
POST /api/providers/mode POST /api/steer	Select an ensemble or add a session restriction.
GET /api/metrics GET /api/executive-brief	Read usage/outcome metrics and HTML/Markdown evidence summaries.
GET /api/sarif WebSocket /ws/telemetry	Export findings and verified fixes; consume replayable live events.

Important limits

Static analysis is bounded: reflection, dynamic dispatch and external dependencies remain review gaps. General steering is not a formal policy language. Docker isolation and kernel traces strengthen evidence but do not prove arbitrary code safe. Provider availability and model responses can vary between runs.

Storage/decode adapters test replay robustness; they do not establish a native second-order persistence flaw. Unsupported scenarios and failed original-code controls are not counted as neutralized. Dry-run outcomes are explicitly simulated and export no verified SARIF fixes. Unknown usage or prices remain unknown.

Useful future work - proposals

Expand harness support and positive controls for additional CWE families and application shapes; add authenticated multi-user deployment and CI review integration; strengthen graph resolution and reproducible evaluation across larger, independent target sets. These are extension directions, not shipped guarantees.

Sources and further reading

Repository snapshot: AgentWard-MuseVerify, commit 99ac51a.
Project and setup: README.md.
Recorded acceptance: docs/VERIFICATION.md.
Implementation: source modules and tests at the same snapshot; see pages 7-8.
Tailscale: installation and Serve usage documentation.

Terms: BYOK = bring your own API key; CWE = Common Weakness Enumeration; SARIF = Static Analysis Results Interchange Format; TOCTOU = time of check to time of use; AST = abstract syntax tree.

Source: README.md; dashboard.py; docs/VERIFICATION.md